



Betriebssysteme und Systemprogrammierung

4. Rust (Part 1)

Michael Schöttner

4.0 Why Rust for OS development?

- Strong type system & many checks at compile time
- Memory Safety → security advantages
- Compile-time garbage collection → fast code
- Rust prevents data races at compile time
- Bare metal access possible where needed (but unsafe)



4.1 Outline

- **Basics**
- Advanced stuff

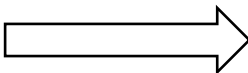
- Defining variables
- Primitive types
- Compound types
- Functions
- If else
- Loops: while, for, loop
- Enums
- Matching
- Impl blocks
- Scopes
- Ownership, borrowing

- The compiler will try to guess the type

```
let x: i32 = 1;  
let x = 2;
```

- Variables are **immutable** by default

```
// This won't compile  
let x = 1;  
x = 2;
```



```
// This is OK  
let mut x = 1;  
x = 2;
```

- Signed integers: `i8`, `i16`, `i32`, `i64`, `i128`
- Unsigned integers: `u8`, `u16`, `u32`, `u64`, `u128`
- Floating point: `f32`, `f64`
- Boolean: `boolean`
- Character: `char` (use single quotes, e.g. `'c'`)

- String literals: `"Rust"` are not type `String` but `&'static str`
 - Are called references to a string slice in ready-only memory
 - Will be discussed in the 2nd part

■ Tuples:

```
// The type annotation is unnecessary  
let my_tuple: (i32, char, bool) = (162, 'X', true);
```

■ Arrays:

```
// Arrays have a fixed size  
let nums: [i32, 5] = [1, 2, 3, 4, 5];  
let second = nums[1];
```

■ Structs:

```
struct Coordinate {  
    x: i32,  
    y: i32,  
}  
  
let point = Coordinate { x: 5, y: 3, };
```

- A function that squares the input:

```
fn square(x: i32) -> i32 {  
    return x * x;  
}
```

- Equivalent to:

```
fn square(x: i32) -> i32 {  
    x * x  
}
```

- If a function does not return a value, just omit the return type:

```
fn add_one(x: &mut i32) {  
    *x += 1;  
}
```


- A function that prints a string:

```
fn main() {  
    println!("Hello world!");  
}
```

- `println!` is a **macro**. The latter always have the exclamation mark as suffix.
- The compiler expands macros during compilation and macros will be replaced by „regular“ Rust source code.

- Rust has conditional if statements. No parentheses for the boolean expressions!

```
let x = 4;  
if x > 5 {  
    println!("Greater than 5");  
} else if x > 3 {  
    println!("Greater than 3");  
} else {  
    println!("x was not big enough");  
}
```

- All expressions can evaluate to a value.

```
let x = 4;
let message = if x > 5 {
    "Greater than 5"
} else if x > 3 {
    "Greater than 3"
} else {
    "x was not big enough"
};
println!("{}", message)
```

- All expressions can evaluate to a value.

```
let x = 4;
let message = if x > 5 {
    "Greater than 5"
} else if x > 3 {
    "Greater than 3"
} else {
    "x was not big enough"
};
println!("{}", message)
```

```
loop {  
    println!("stuck in a loop");  
}
```

- Use break to exit

```
loop {  
    break;  
}
```

- Loop expressions can evaluate to a value, just like any other expression:

```
let mut count = 0;  
let three = loop {  
    count += 1;  
    if count >= 3 {  
        break count;  
    }  
};
```

- Rust while loops are fairly straightforward:

```
let mut count = 5;  
while count > 0 {  
    count -= 1;  
}
```

- A `for` loop can iterate over a collection

```
let nums = [0, 1, 2, 3, 4];  
for n in nums {  
    println!("{}", n);  
}  
// Prints 0 1 2 3 4
```

- Range notion:

```
for n in 0..5 {  
    println!("{}", n);  
}  
// Prints 0 1 2 3 4
```

More primitive types: `usize` and `isize`

- On a 32-bit (x86) system: `usize` is equivalent to `u32`
- On a 64-bit (x86_64) system: `usize` is equivalent to `u64`
- The signed counterpart is `isize`, which follows the same rule: it has the same number of bits as a pointer on the target architecture, but is a signed integer type (with sign).
- `usize` and `isize` are used mainly for:
 - Indexing arrays
 - Representing memory sizes and offsets
- Example:
 - `arr.len()` returns a `usize`
 - `i` is also of type `usize`

```
let arr = [10, 20, 30];  
  
for i in 0..arr.len() {  
    println!("{}", arr[i]);  
}
```


- Useful when a type should have only a few possible values

```
enum Coin {  
    Head,  
    Tail,  
}
```

```
let a = Coin::Head;  
let b = Coin::Tail;
```

- Each value in an `enum` is called a **variant**.

Enums (2)

■ Enums can also store data!

```
#[derive(Debug)]  
#[allow(dead_code)]  
enum OperatingSystem {  
    Mac,  
    Windows,  
    Linux,  
    Other(String),  
}  
  
fn main() {  
    let a = OperatingSystem::Linux;  
    let b = OperatingSystem::Other("Redox OS".to_string());  
  
    println!("Operating System A: {:?}", a);  
    println!("Operating System B: {:?}", b);  
}
```

Attribute telling compiler to automatically generate the debug trait, used for formatted output (see later)

enum.rs

Attribute telling compiler to suppress warnings because of unused items and that the String in Other is never accessed.

Formatted output using debug trait which is automatically derived, see above.

- Rust **match expressions** are like C switch statements. However, they must *always be exhaustive*.
- What is wrong here?

```
let num = 162;  
  
match num {  
    160 => println!("{}", 160);  
    161 => println!("{}", 161);  
    168 => println!("{}", 168);  
}
```

- The match is not exhaustive! What if num was 164 or 10?

- This **is** valid:

```
let num = 162;  
  
match num {  
    160 => println!("{}", 160);  
    161 => println!("{}", 161);  
    168 => println!("{}", 168);  
    _ => println!("{}", "another case");  
}
```

- The underscore matches anything that was not already matched.
- Each pattern in the match statement is called a **match arm**.

- Matching is very useful in combination with enums. Match expressions can also evaluate to a value (just like any other expression).

```
enum OperatingSystem {  
    Mac,  
    Windows,  
    Linux,  
    Other(String),  
}  
  
fn os_name(os: OperatingSystem) -> String {  
    match os {  
        OperatingSystem::Mac      => "mac".to_string(),  
        OperatingSystem:: Windows => "windows".to_string(),  
        OperatingSystem:: Linux   => "linux".to_string(),  
        OperatingSystem:: Other(s) => s,  
    }  
}
```

match.rs

- **Scopes** are enclosed in curly braces:

```
fn do_stuff() {  
    let a = String::from("hello");  
    {  
        let b = String::from("goodbye");  
        // Can access a and b  
        // ...  
        // b goes out of scope and is dropped  
    }  
    // Cannot access b here: it is out of scope  
    // a is dropped at the end of the function  
}
```

- Functions, loops, if statements etc. have their own scope.
- You can also create nested scopes using curly braces.

- Suppose we have the following struct definition:

```
struct Vector {  
    x: f64,  
    y: f64,  
}
```

- We might want to add 2 Vectors elementwise. Here is one way to do that:

```
fn add(v1: Vector, v2: Vector) -> Vector {  
    x: v1.x + v2.x,  
    y: v1.y + v2.y,  
}
```

- We can also do the same thing using `impl` blocks, which define methods related to a `struct` (or `enum`).

```
impl Vector {  
    fn add(&self, other: Vector) -> Self {  
        x: self.x + other.x,  
        y: self.y + other.y,  
    }  
}  
  
let sum = v1.add(v2);
```

- `Self` is a shorthand for the type of the `impl` block (in this case, `Vector`).
- `self` refers to the instance the method is called on.

impl blocks for compound types

- Now first argument is `&mut self` allowing us to mutate the object

```
impl Vector {  
    fn double(&mut self) {  
        self.x *= 2;  
        self.y *= 2;  
    }  
}
```

- Rust strictly distinguishes between read and write access!

- A very important concept
- Used to ensure:
 - Memory safety
 - Compile-time garbage collection
 - Data-race-free concurrency by default
- At the beginning sometimes difficult

What is wrong here?

```
struct Vector {  
    x: f64,  
    y: f64,  
}
```

vector1.rs

```
fn add(v1: Vector, v2: Vector) -> Vector {  
    Vector {  
        x: v1.x + v2.x,  
        y: v1.y + v2.y,  
    }  
}
```

```
fn main() {  
    let v1 = Vector {x: 1.0, y: 2.0};  
    let v2 = Vector {x: 3.0, y: 4.0};  
    let v3 = add(v1, v2);  
  
    println!("Result: ({} , {})", v3.x, v3.y);  
    println!("Result: ({} , {})", v1.x, v1.y);  
}
```

What is wrong here?

```
fn add(v1: Vector, v2: Vector) -> Vector {  
    Vector {  
        x: v1.x + v2.x,  
        y: v1.y + v2.y,  
    }  
}
```

```
fn main() {  
    let v1 = Vector {x: 1.0, y: 2.0};  
    let v2 = Vector {x: 3.0, y: 4.0};  
    let v3 = add(v1, v2);  
  
    println!("Result: ({} , {})", v3.x, v3.y);  
    println!("Result: ({} , {})", v1.x, v1.y);  
}
```

vector1.rs

error[E0382]: borrow of moved value: `v1`

--> vector.rs:19:39

```
14 |     let v1 = Vector {x: 1.0, y: 2.0};  
    |         -- move occurs because `v1` has type `Vector`, which does not implement the `Copy` trait  
15 |     let v2 = Vector {x: 3.0, y: 4.0};  
16 |     let v3 = add(v1, v2);  
    |                 -- value moved here  
...  
19 |     println!("Result: ({} , {})", v1.x, v1.y);  
    |                                   ^^^^ value borrowed here after move
```

- Instantiating a type and **binding** it to a variable name creates a memory resource that the Rust compiler will validate through its whole **lifetime**.
- The bound variable is called the resource's **owner**.
- Our example:

```
fn main() {  
    // Below we instantiate a struct Vector  
    // and bind it to variable 'v1' to create a memory resource  
    let v1 = Vector {x: 1.0, y: 2.0};  
    // 'v1' is the owner
```

- Sometimes that owner can change. This is called a **move**
- Assignment moves values.
- This is invalid:

```
let s1 = String::from("my string");  
let s2 = s1;           // Ownership of the string moves from s1 to s2.  
  
// s1 no longer owns the string, so we can't access data via s1.  
println!("{}", s1); // This is an error; data has been moved out of s1.
```

- This is fine:

```
let s1 = String::from("my string");  
let s2 = s1;           // Ownership of the string moves from s1 to s2.  
println!("{}", s2); // This is okay; s2 owns the string now.
```

- Passing a value to a function moves the value

```
fn main() {  
    let s = String::from("hello");  
    do_stuff(s);  
    // s no longer accessible; it was moved into do_stuff  
}  
  
fn do_stuff(s: String) {  
    // do stuff with s  
}
```

- Returning a value from a function moves the value back to the caller:

```
fn main() {  
    let s = get_string();  
    // We can now use s, which owns the string  
}  
  
fn get_string() -> String {  
    String::from("hello")  
}
```


Recap

```
fn add(v1: Vector, v2: Vector) -> Vector {  
    Vector {  
        x: v1.x + v2.x,  
        y: v1.y + v2.y,  
    }  
}
```

```
fn main() {  
    let v1 = Vector {x: 1.0, y: 2.0};  
    let v2 = Vector {x: 3.0, y: 4.0};  
    let v3 = add(v1, v2);  
  
    println!("Result: ({} , {})", v3.x, v3.y);  
    println!("Result: ({} , {})", v1.x, v1.y);  
}
```

vector1.rs

error[E0382]: borrow of moved value: `v1`

--> vector.rs:19:39

```
14 |     let v1 = Vector {x: 1.0, y: 2.0};  
    |         -- move occurs because `v1` has type `Vector`, which does not implement the `Copy` trait  
15 |     let v2 = Vector {x: 3.0, y: 4.0};  
16 |     let v3 = add(v1, v2);  
    |                 -- value moved here  
...  
19 |     println!("Result: ({} , {})", v1.x, v1.y);  
    |                                   ^^^^ value borrowed here after move
```

- If you need multiple variables to own data, you can clone a value:

```
let s1 = String::from("my string");  
let s2 = s1.clone(); // s2 is a clone of s1  
println!("{}", s2);  // This is okay; s2 owns its data.  
println!("{}", s1);  // This is okay; s1 also owns its data.
```

- Note that cloning is usually *expensive*.
In this case, we are allocating memory *twice*.
- Cloned values are completely independent of the value they were cloned from.
If *s1* is modified, code using *s2* will not see those changes.

- There is one case in which a move behaves like a clone: when the type is Copy.
- Copy is a **trait** (traits see later) that indicates that a value is copied whenever it is used.
- For example, integers are Copy:

```
let x = 5;  
let y = x;  
println!("{}", x, y);
```

- This is fine because the value in `x` (5) is **copied**, not moved.
So `x` and `y` both own their values and can be accessed.

- In general, types that require heap allocations are not `Copy`.
- `Copy`: integers, floats, booleans, chars, immutable references, and compound types containing only `Copy` types.
- Not `Copy`: Strings, Vectors, mutable references, and compound types containing at least one non-`Copy` type.

- Consider the following struct:

```
struct Foo {  
    x: i32,  
}
```

copy.rs

- Is it Copy? **No**. But shouldn't it be Copy, since it only contains Copy types?
- We must explicitly tell the Rust compiler we wish to make it Copy
- Copy always implies Clone (Clone is a supertrait of Copy)

```
#[derive(Clone)]  
#[derive(Copy)]  
struct Foo {  
    x: i32,  
}
```

Copy is used implicitly
Clone is used explicitly

- Rust uses the end of scope as the place to deallocate a resource.
- The term for this deallocation is called a **drop**.

```
struct Foo {  
    x: i32,  
}  
  
fn main() {  
    // We instantiate structs and bind to variables  
    // to create memory resources  
    let foo = Foo { x: 42 };  
    // 'foo' is the owner  
  
    println!("{}", foo.x);  
    // 'foo' is dropped here  
}
```

- When a struct is dropped, the struct itself is dropped first, then its children are dropped individually, and so on.

```
struct Bar {  
    x: i32,  
}  
  
struct Foo {  
    bar: Bar,  
}  
  
fn main() {  
    let foo = Foo { bar: Bar { x: 42 } };  
    println!("{}", foo.bar.x);  
    // 'foo' is dropped first  
    // then 'foo.bar' is dropped  
}
```

- When a value's owner goes out of **scope**, the value is automatically **dropped** (= freed)
- To manually drop a value v *early*, call `drop(v)`
- Is it possible to drop a value late?
- No. A value is dropped when it goes out of scope. You can modify your code so that the scope is larger, but you cannot call `drop(v)` if v is not in scope.
- Remark: dropping is a compile-time garbage collection

- **References let you access data by borrowing without taking ownership of it.**
- The borrow checker enforces Rust's ownership and borrowing rules to guarantee memory safety at compile time - without needing a garbage collector.

- Rust **references** are like pointers in C, but are:
 - Always valid (no pointers to freed memory)
 - Never null
- The Rust compiler checks these properties at **compile time**
- Reference with `&`, dereference with `*`

```
let x = 2;  
let ptr = &x;  
println!("{}", *ptr);
```

- To obtain a mutable reference, the variable itself must be mutable:

```
let x = 2;  
let ptr = &mut x; // Cannot mutably reference an immutable variable
```

```
let mut x = 2;  
let ptr = &mut x; // This is correct
```



- Obtaining a (mutable) reference in Rust is called **borrowing**.

Borrowing ownership with references

- References allow us **borrow read-only access** to a resource with the **&** operator.
- References are also dropped like other resources.
- Example:
 - We can access the instantiated struct through `foo` and `f`

```
#[derive(Debug)]
struct Foo {
    x: i32,
}

fn main() {
    let foo = Foo { x: 42 };
    let f = &foo;
    println!("{}", f.x);
    println!("{}", foo.x);
    // 'f' is dropped here
    // 'foo' is dropped here
}
```

borrow.rs

Borrowing ownership with mutable references

- We can also **borrow mutable access** to a resource with the **&mut** operator.
- A resource owner cannot be moved or modified while mutably borrowed.
- Reason: Rust prevents having two ways to mutate an owned value because it introduces the possibility of a data race.
- Example:

```
struct Foo {  
    x: i32,  
}  
  
fn do_something(f: &mut Foo) {  
    println!("{}", f.x);  
    f.x += 1;  
    // 'f' is dropped here  
}
```

```
fn main() {  
    let mut foo = Foo { x: 42 };  
    let f = &mut foo;  
    let g = &mut foo;  
  
    do_something(f);  
    do_something(g);  
}
```

mut-borrow.rs



■ Example: done right

```
struct Foo {  
    x: i32,  
}  
  
fn do_something(f: Foo) {  
    println!("{}", f.x);  
    f.x += 1;  
    // 'f' is dropped here  
}
```

```
fn main() {  
    let mut foo = Foo { x: 42 };  
  
    do_something(&mut foo);  
    do_something(&mut foo);  
}
```

mut-borrow.rs



- Dereferencing is done using the `*` operator
 - You can read/write the owner's value
 - Or you get a copy of an owned value, if the value has copyable type

```
fn main() {  
    let mut val1 = 42;  
    let ref1 = &mut val1;  
    let val2 = *ref1; // get a copy of the owner's value  
    *ref1 = 13; // set the reference's owner's value  
    println!("{}", val2);  
    println!("{}", val1);  
}
```



deref.rs

```
MBP22:tmp mschoett1$ ./test  
42  
13
```

Dereferencing for non-copyable types

- Dereferencing is done using the `*` operator
- **The value is moved, if the value has a non-copyable type!, e.g. struct types**

```
#[derive(Debug)]
struct Foo {
    x: i32,
}

fn main() {
    let mut foo1 = Foo { x: 42 };
    let ref1 = &mut foo1;
    let foo2 = *ref1; // move required
    (*ref1).x = 13; // set the reference's owner's value
    println!("{:?}", foo2);
    println!("{:?}", foo1);
}
```

deref2.rs



Dereferencing for non-copyable types

- Dereferencing is done using the `*` operator
- The value is moved, if the value has a non-copyable type!, e.g. struct types

```
#[derive(Debug)]  
#[derive(Clone)]  
#[derive(Copy)]  
struct Foo {  
    x: i32,  
}  
  
fn main() {  
    let mut foo1 = Foo { x: 42 };  
    let ref1 = &mut foo1;  
    let foo2 = *ref1; // implicit copy  
    (*ref1).x = 13; // set the reference's owner's value  
    println!("{:?}", foo2);  
    println!("{:?}", foo1);  
}
```

deref2.rs

Dereferencing for non-copyable types

- `Copy` is implicit, inexpensive, and cannot be re-implemented.
- `Clone` is explicit, may be expensive, and may be re-implemented.

Q1: What is wrong here?

```
fn main() {  
    let v1 = Vector {x: 1.0, y: 2.0};  
    let v2 = Vector {x: 3.0, y: 4.0};  
    let v3 = add(v1, v2);  
  
    println!("Result: ({} , {})", v3.x, v3.y);  
    println!("Result: ({} , {})", v1.x, v1.y);  
}
```

Q1: What is wrong here?

```
fn main() {  
    let v1 = Vector {x: 1.0, y: 2.0};  
    let v2 = Vector {x: 3.0, y: 4.0};  
    let v3 = add(v1, v2);  
  
    println!("Result: ({}", v3.x, v3.y);  
    println!("Result: ({}", v1.x, v1.y);  
}
```

error[E0382]: borrow of moved value: `v1`

→ vector.rs:19:39

```
14 |     let v1 = Vector {x: 1.0, y: 2.0};  
    -- move occurs because `v1` has type `Vector`, which does not implement the `Copy` trait  
15 |     let v2 = Vector {x: 3.0, y: 4.0};  
16 |     let v3 = add(v1, v2);  
    -- value moved here  
...  
19 |     println!("Result: ({}", v1.x, v1.y);  
    ^^^^^ value borrowed here after move
```

Q1: What is wrong here?

```
struct Vector {  
    x: f64,  
    y: f64,  
}
```

Solution: borrowing v1 and v2

```
fn add(v1: &Vector, v2: &Vector) -> Vector {  
    Vector {  
        x: v1.x + v2.x,  
        y: v1.y + v2.y,  
    }  
}
```

```
fn main() {  
    let v1 = Vector {x: 1.0, y: 2.0};  
    let v2 = Vector {x: 3.0, y: 4.0};  
    let v3 = add(&v1, &v2);  
  
    println!("Result: ({} , {})", v3.x, v3.y);  
    println!("Result: ({} , {})", v1.x, v1.y);  
}
```

Q2: What is wrong here?

```
struct Vector {  
    x: f64,  
    y: f64,  
}  
  
impl Vector {  
    fn double(&mut self) {  
        self.x *= 2.0;  
        self.y *= 2.0;  
    }  
}
```

```
fn main() {  
    let mut v1 = Vector { x: 1.0, y: 2.0 };  
  
    let v2 = &v1;  
  
    v1.double();  
    println!("borrow = ({{}}, {{}})", v2.x, v2.y);  
}
```

Q2: What is wrong here?

```
impl Vector {  
    fn double(&mut self) {  
        self.x *= 2.0;  
        self.y *= 2.0;  
    }  
}
```

```
fn main() {  
    let mut v1 = Vector { x: 1.0, y: 2.0 };  
  
    let v2 = &v1;  
  
    v1.double();  
    println!("borrow = ({}, {})", v2.x, v2.y);  
}
```

error[E0502]: cannot borrow `v1` as mutable because it is also borrowed as immutable
--> vector.rs:32:14

```
30 |     let v2 = &v1;  
    |           --- immutable borrow occurs here  
31 |  
32 |     let v3 = v1.double();  
    |           ^^^^^^^^^^^^^ mutable borrow occurs here  
33 |  
34 |     println!("borrow = ({}, {})", v2.x, v2.y);  
    |                                   ---- immutable borrow later used here
```

Q3: What is wrong here?

```
struct Vector {  
    x: f64,  
    y: f64,  
}  
  
impl Vector {  
    fn double(&mut self) {  
        self.x *= 2.0;  
        self.y *= 2.0;  
    }  
}
```

```
fn main() {  
    let mut v1 = Vector { x: 1.0, y: 2.0 };  
  
    // Mutable borrow of v1  
    let v2 = &mut v1;  
  
    v1.double();  
    println!("borrow = ({{}, {{}})", v2.x, v2.y);  
}
```


Q3: What is wrong here?

```
impl Vector {  
    fn double(&mut self) {  
        self.x *= 2.0;  
        self.y *= 2.0;  
    }  
}
```

```
fn main() {  
    let mut v1 = Vector { x: 1.0, y: 2.0 };  
  
    // Mutable borrow of v1  
    let v2 = &mut v1;  
  
    v1.double();  
    println!("borrow = ({{}, {{}})", v2.x, v2.y);  
}
```

error[E0499]: cannot borrow `v1` as mutable more than once at a time

→ vector.rs:32:14

30 | let v2 = &mut v1;

----- first mutable borrow occurs here

31 |

32 | let v3 = v1.double();

^^ second mutable borrow occurs here

33 |

34 | println!("borrow = ({{}, {{}})", v2.x, v2.y);

---- first borrow later used here

- **You can have one mutable reference, or multiple immutable references. But not both.**
- References are always valid and non-null.
- Every value has one owner.
- Values are freed when their owner goes out of scope.
- Assignment moves values (unless the value is `Copy`, see later).
- A reference can never outlive its value (see later)